

Project Guidelines

You get a demo script that performs a DPA style attack on a set of power traces.

The demo is based on a differential attack, using a Hamming weight leakage model, and correlation as a distinguisher.

Your job is to investigate several questions:

1. rewrite the script so that you are attacking bits instead of bytes, using
 - a. a single bit leakage model
 - b. using both correlation as well as a difference of means based distinguisher

Does the choice of distinguisher make any difference?

2. rewrite the script so that you are attacking the input of the SubBytes operation, and attack this intermediate with a distinguisher of your choice.
What do you notice in terms of attack outcomes?
3. how many traces are needed at the least (for your best attack strategy on these traces) to determine the AES key with reasonable enumeration effort?

1. Attack Bits instead of Bytes

In this case, we use the output of the S-Box operation as intermediate variable.

Note that to target something that is non linear is better than doing it with something that is linear.

So working with the Sub-bytes is a good choice.

This because we want to exploit data dependent differences across different power traces.

For calculating the hypothetical power values, the given code uses an Hamming Weight power model. It could be really useful for our objective because in some processors the bus still hold the value of the previous bus transfer.

For each of the 16 Sub Bytes (because we work with 16 bytes) we have $2^8 = 256$ possible key hypothesis.

This is the main that I used to work, inside the report we will go along all used functions.

```
252 if __name__ == '__main__':
253
254     ##### Initialize section ##### (uncomment just one of them)
255
256     # 1) This one works on bytes and attacks the SubBytes operation output
257     #[trs, Hwguess] = AESAttack().Initialise(1000)
258
259     # 2) This one works on bits and attacks the SubBytes operation output
260     [trs, Hwguess] = AESAttack().Initialise_bit(1000)
261
262     # 3) This one works on bytes and attacks the SubBytes operation input, so the ARK operation output
263     #[trs, Hwguess] = AESAttack().Initialise_only_ADK(1000)
264
265
266     plt.ion()
267     fig = plt.figure()
268     ax = []
269     for byteno in range(16):
270         ax.append(fig.add_subplot(4, 4, byteno+1))
271
272     for Nm in range(1, 10):
273         fig.suptitle('N={0}'.format(Nm * 100))
274         print("N={0}\n".format(Nm * 100))
275
276         for byteno in range(16):
277             print("byte={0}\n".format(byteno))
278
279             ##### Statistical analysis section ##### (uncomment just one of them)
280
281             # 0) Correlation as distinguisher, target bytes
282             #AESAttack().corrAttack(trs, ax[byteno], byteno, Nm*100)
283
284             # 1) Correlation as distinguisher, target bits
285             AESAttack().corrAttack_bit(trs, ax[byteno], byteno, Nm*100)
286
287             # 2) Difference of Means as distinguisher, target bits
288             #AESAttack().dom_attack_bit(trs, ax[byteno], byteno, Nm*100)
289
290             fig.canvas.draw()
291             fig.canvas.flush_events()
292             plt.show()
293             plt.tight_layout()
294             plt.pause(.001)
295
296     plt.ioff()
297     plt.show()
```

1.a. Use a single bit leakage model

To operate on bit level, we need to loop one the 16 bytes and do the AES ARK and SubBytes operations for each of the 256 key hypothesis.

In this way we can obtain the output of each SubBytes and so loop again, but this time on the 8 bits of the byte under analysis.

Given a binary number Y and shifting bits, we can obtain each single bit value.

I used the function **shift and get lsb(Y, bit pos)** to do this.

There, using the right shift operator, we can translate bits of the input number and obtain the one in which we are interested.

In this way we can set our guesses.

Note that “ ... & 1 ” is necessary to select the least significant bit.

This function get used inside **Initialise bit(self, N)**

```

128     def shift_and_get_lsb(self, X, bit_position):
129         '''This function does the right shift of bits composing X by a number o positions
130         The AND with 1 (in binary) let us obtain the least significant value that is the bit
131         in which we are interested in
132         '''
133         return (X >> bit_position) & 1
134
135     def Initialise_bit(self, N):
136         '''This funtion reads the traces from file and returns the tridimensional array HWguess that contains
137         our guesses about that bit in that byte about that key guess
138         '''
139         trs = TRS_Reader.TRS_Reader("TinyAES_625Samples_FirstRoundSbox.trs")
140         trs.read_header()
141         trs.read_traces(N, 0, trs.number_of_samples)
142
143         # Because we decided to work on bit instead of bytes, we add a new dimension that works on the 8 bits
144         HWguess = np.zeros((16, 256, 8, N)).astype(int)
145
146         for byteno in range(16):
147             X = trs.plaintext[:, byteno]
148             print("byteno={0}\n".format(byteno))
149             for kg in range(256):
150                 Y = AESAttack().ADK(X, kg)
151                 Y = AESAttack().Sbox(Y)
152                 # Now go along all the single 8 bits
153                 for bit_pos in range(8):
154                     HWguess[byteno, kg, bit_pos] = AESAttack().shift_and_get_lsb(Y, bit_pos)
155
156         return [trs, HWguess]

```

Inside the new correlation attack on bit level **corrAttack_bit(self, trs, ax, byteno, Nm)** we compute the Pearson correlation index using the same “maxCorr” function defined in the given code.

We want to get array of correlations “corr” which is an array that stores correlation values for each time sample and the max correlation value.

Then the code is similar as before.

```

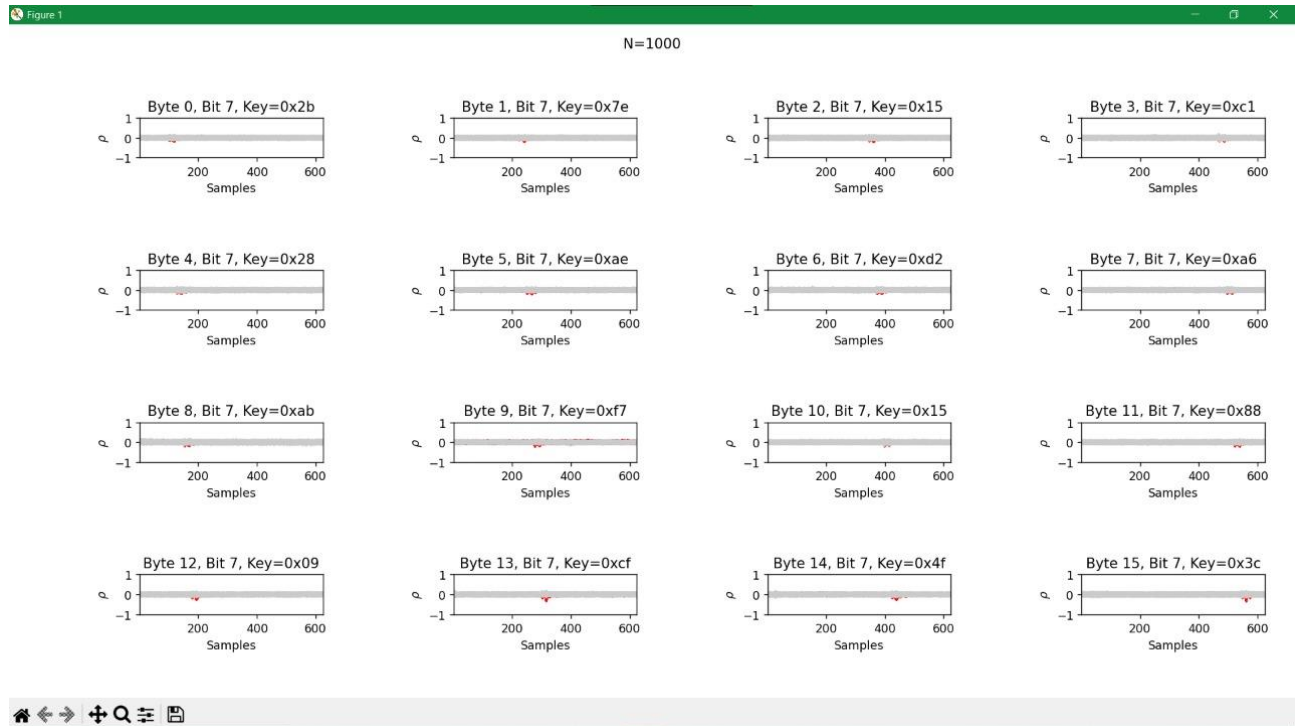
158     def corrAttack_bit(self, trs, ax, byteno, Nm):
159         '''This function performs the correlation attack using the maxCorr function'''
160
161         # Nm are the number of traces to utilize
162         maxkg = 0
163         maxcorr_k = 0
164         for kg in range(256):
165             for bit_pos in range(8):
166                 bit_values = HWguess[byteno, kg, bit_pos]
167                 maxcorr, corr = AESAttack.maxCorr(self, bit_values[:Nm], trs.traces[:Nm, :])
168                 if maxcorr > maxcorr_k:
169                     maxkg = kg
170                     maxcorr_k = maxcorr
171                 if kg == key[byteno]:
172                     ax.plot(corr, 'r-', alpha=1)
173                 else:
174                     ax.plot(corr, color=(0.8, 0.8, 0.8), alpha=0.8)
175             ax.set_xlim([1, trs.number_of_samples])
176             ax.set_ylim([-1, 1])
177             ax.title.set_text(f'Byte {byteno}, Bit {bit_pos}, Key=0x{maxkg:02x}')
178             ax.set_xlabel('Samples')
179             ax.set_ylabel(r'$\rho$')
180         return maxkg

```

Executing this code took more than 4 hours of work on my machine.

Unfortunately, the byte number 3 is the only problematic one, maybe there is some sort of statistical noise that confuses our max correlation estimation.

All other bytes are correct, here is the output plot for the execution with 1000 traces.



1.b. Difference of means based distinguisher

The idea here is to implement the calculation of the t-statistic using this formula:

$$t = \frac{\text{mean}(\text{orange}) - \text{mean}(\text{blue})}{\sqrt{\frac{\text{var}(\text{orange})}{N_{\text{orange}}} + \frac{\text{var}(\text{blue})}{N_{\text{blue}}}}}$$

The code can be found inside **calculate t statistic(self, trs, leakage)**

```

182     def calculate_t_statistic(self, trs, leakage):
183         """
184         This function calculates the t-statistic using the formula inside the provided report.
185         Also identifies the point (sample i) where the difference between the two classes is
186         statistically more significant.
187         """
188
189         # trs is the matrix of traces, where each line represents a single acquisition and each
190         # column represents a sample of that track
191
192         # variable that we will give in output
193         max_difference = 0
194
195         # Split into groups
196         group_0 = trs[leakage == 0, :]
197         group_1 = trs[leakage == 1, :]
198
199         group_0_means = np.mean(group_0, axis=0)
200         group_1_means = np.mean(group_1, axis=0)
201
202         # Obtain the variance
203         group_0_variance = np.var(group_0, axis=0, ddof=1)
204         group_1_variance = np.var(group_1, axis=0, ddof=1)
205
206         # Number of samples for group 0 and group 1
207         # 0 is used to return only the number of rows
208         n0 = group_0.shape[0]
209         n1 = group_1.shape[0]
210
211         # Standard Deviation
212         standardError = np.sqrt(group_0_variance / n0 + group_1_variance / n1)
213         standardError[standardError == 0] = np.inf
214
215         t_statistic = (group_1_means - group_0_means) / standardError
216
217         # Calculate the absolute maximum value of the t-statistic vector
218         # Using the absolute value because the t-statistic can be positive or negative
219         max_difference = np.max(np.abs(t_statistic))
220
221         return max_difference, t_statistic

```

To use a difference of means based (DoM) distinguisher means confronting the mean value of 2 groups (here denoted as orange and blue).

So split into 2 groups and distinguish if the bit guess is set to 1 or 0.

Then let's calculate

- the mean vector for both groups
- the vector of variances for both groups
- the number of elements that each group is composed of
- the vector of standard deviations
- the t-statistic, which is given by the difference of the means of the two groups divided by the corresponding standard error

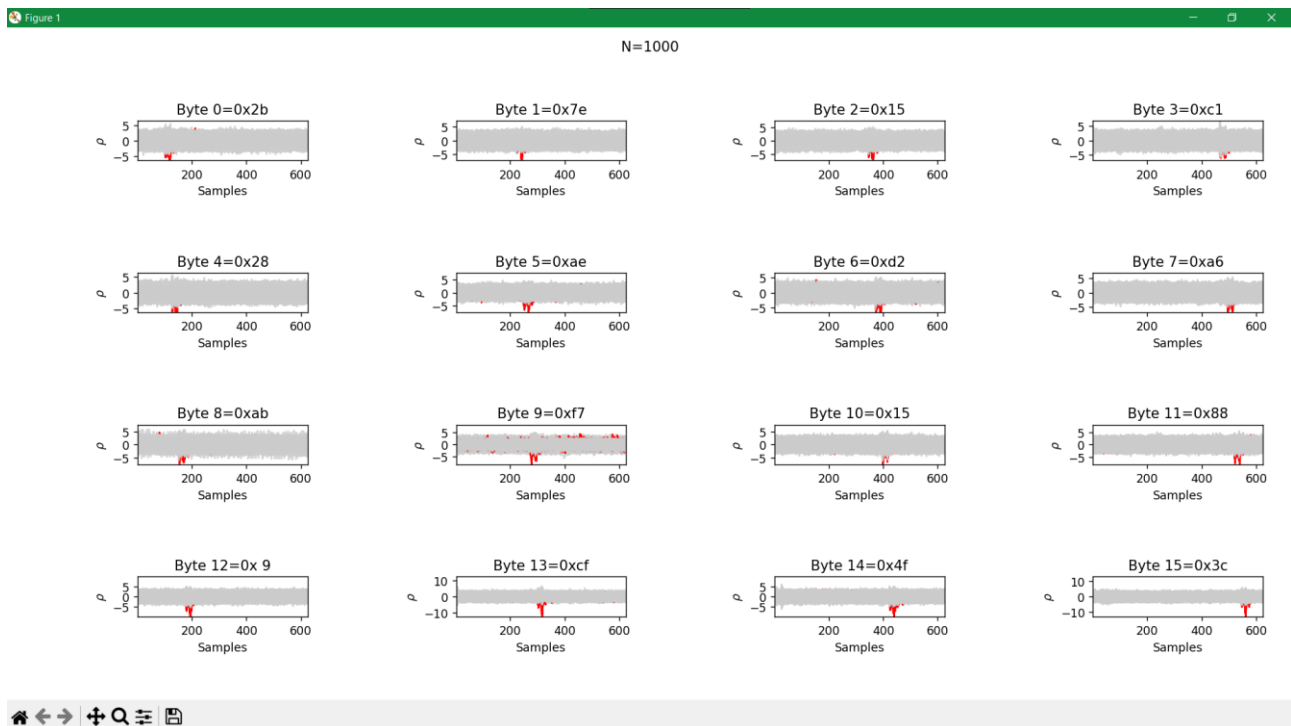
This process is repeated for all samples in the traces.

We return the maximum value of the difference of means for this key guess and the t-statistic.

Then inside **dom_attack_bit(self, trs, ax, byteno, Nm)** we proceed to execute the attack for each bit of the various bytes.

```
223 def dom_attack_bit(self, trs, ax, byteno, Nm):
224     ...
225     This function is structured in a similar way as corrAttack_bit, but here we use the DoM as distinguisher
226     ...
227     ax.clear()
228     maxkg = 0
229     maxmean_k = 0
230     for kg in range(256):
231         for bit_position in range(8):
232             # Take the respective bit guess from the tridimensional array obtained HWguess from Initialise_bit
233             leakage_bit_value = HWguess[byteno, kg, bit_position]
234             [max_DoM, t_statistic] = AESAttack().calculate_t_statistic(trs.traces[0:Nm, :], leakage_bit_value[:Nm])
235
236             if (max_DoM > maxmean_k):
237                 maxkg = kg
238                 maxmean_k = max_DoM
239             if (kg == key[byteno]):
240                 ax.plot(t_statistic, 'r-', alpha=1)
241             else:
242                 ax.plot(t_statistic, color=(0.8, 0.8, 0.8), alpha=0.8)
243         ax.set_xlim([1, trs.number_of_samples])
244         ax.set_ylim([maxmean_k * (-1), maxmean_k * 1])
245         ax.title.set_text('Byte {0}=0x{1:2x}'.format(byteno, maxkg))
246         ax.set_xlabel('Samples')
247         ax.set_ylabel(r'$\rho$')
248     return maxkg
249
```

The byte number 3 is the only problematic again, all other bytes are correct and here is the output plot for the execution with 1000 traces.



Wrong output bytes : byte3

Number of correct output bytes : 15/16

Does the choice of distinguisher make any difference?

In this case it makes a lot of difference in terms of computation time.

If we decide to use a DoM distinguisher, then the required time is just half an hour.

2. Attack Sub-Bytes input with DoM distinguisher

To do this, we need to comment one line of the given code inside **Initialise(self, N)**

For this purpose, there is a new function called **Initialise_only_ADK(self, N)**

```
83     def Initialise_only_ADK(self, N):
84         ...
85         This one is equivalent to the Initialise one, but here we attack the input of the SubBytes operation
86         ...
87         trs = TRS_Reader.TRS_Reader("TinyAES_625Samples_FirstRoundSbox.trs")
88         trs.read_header()
89         trs.read_traces(N, 0, trs.number_of_samples)
90         print(trs.number_of_samples)
91         HWguess = np.zeros((16, 256, N)).astype(int)
92
93         for byteno in range(16):
94             X = trs.plaintext[:, byteno]
95             print("byteno={0}\n".format(byteno))
96             for kg in range(256):
97                 Y = AESAttack().ADK(X, kg)
98                 # Commented line, we are interested in the input of Sbox, so the output of the ARK
99                 # Y = AESAttack().Sbox(Y)
100                 HWguess[byteno, kg] = AESAttack().HW(Y)
101         return [trs, HWguess]
```

Attacking the input of the Sub Bytes operation consists in attacking the output of the Add Round Key operation.

The idea here is simply to not execute the SubByte operation.

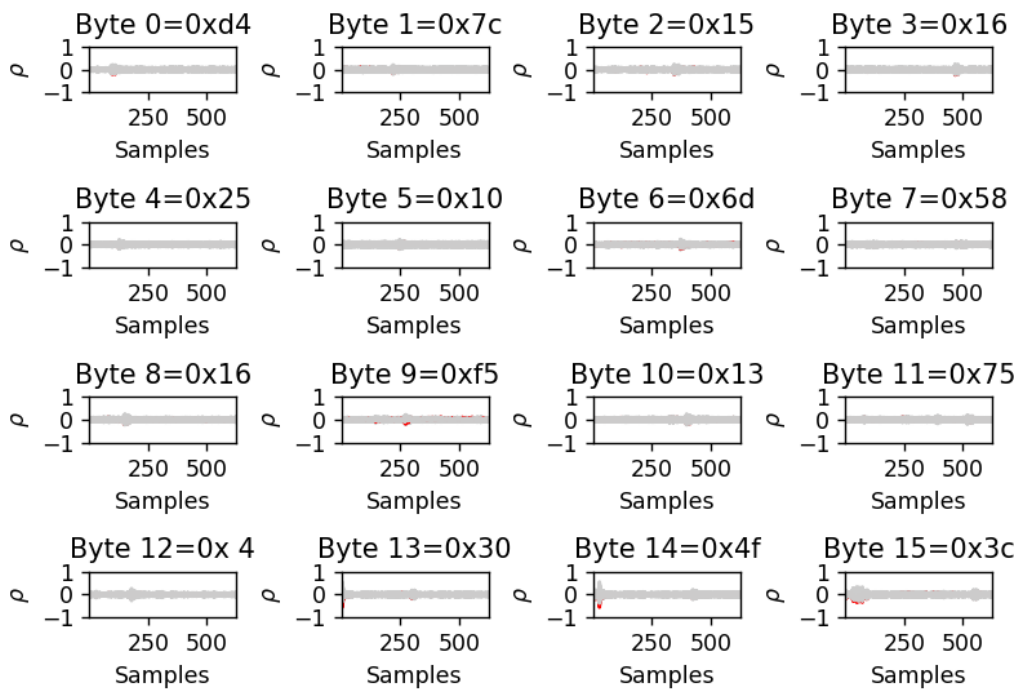
I decided to do the DPA attack using the difference of means distinguisher because of time effort required by other distinguisher approach.

The result of the operation showed that the attack can find only a little of the correct key bytes.

This problem is due to the fact that ARK is a linear function of key and plaintext.

It is more difficult to use such a function to get useful information about leakage.

N=1000



Wrong output bytes : byte0, byte1, byte4, byte5, byte6, byte7, byte8, byte9, byte10, byte11, byte12, byte13

Number of correct output bytes : 4/16

3. Number of needed traces to determine the AES key with reasonable enumeration effort

In first place, I decided to use a different number of traces to see how the output would improve.

Here you can find a table with computed outputs in the case of working on bits and using DoM distinguisher.

DoM Table

Number of used traces	Number of Correct Key Bytes
100	1 / 16
400	11 / 16
500	11 / 16
600	14 / 16
700	15 / 16
800	15 / 16
1000	15 / 16

You can also find screenshots at the end of the document which attest to the above.

It seem that for the DoM distinguisher, if we are using between 600 and 700 traces we obtain already the best performance.

Doing some other tests, I found that the first time we get 15/16 is using between 660 and 670 traces.

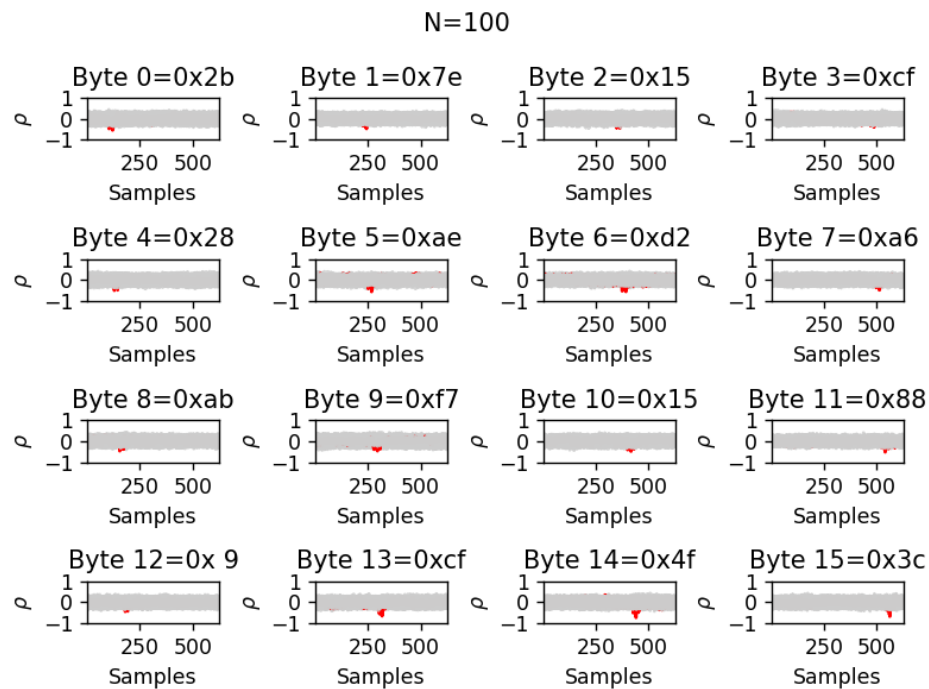
I also decided to try and find the minimum number of traces required using the correlation distinguisher.

In this case, even with just 100 traces we obtain a really good output.

So we can conclude that using a:

- Correlation distinguisher require more time but less traces to succeed
- DoM distinguisher require less time but really much more traces to succeed

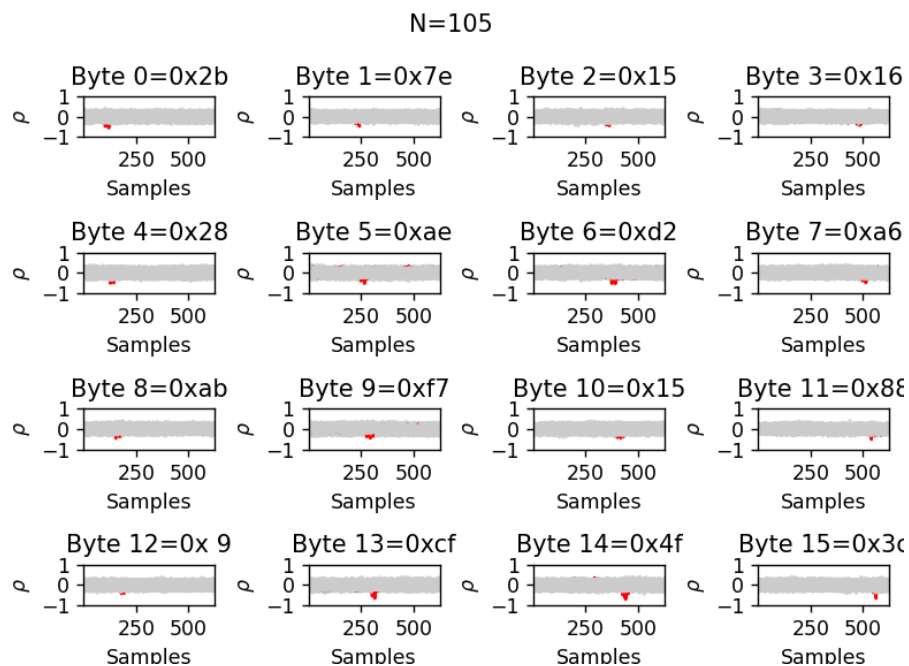
This is the output using just 100 traces and the original code given by the professor for the project.



Wrong output bytes : byte3

Number of correct output bytes : 15/16

If we try again but with 105 traces, we get a successful result.

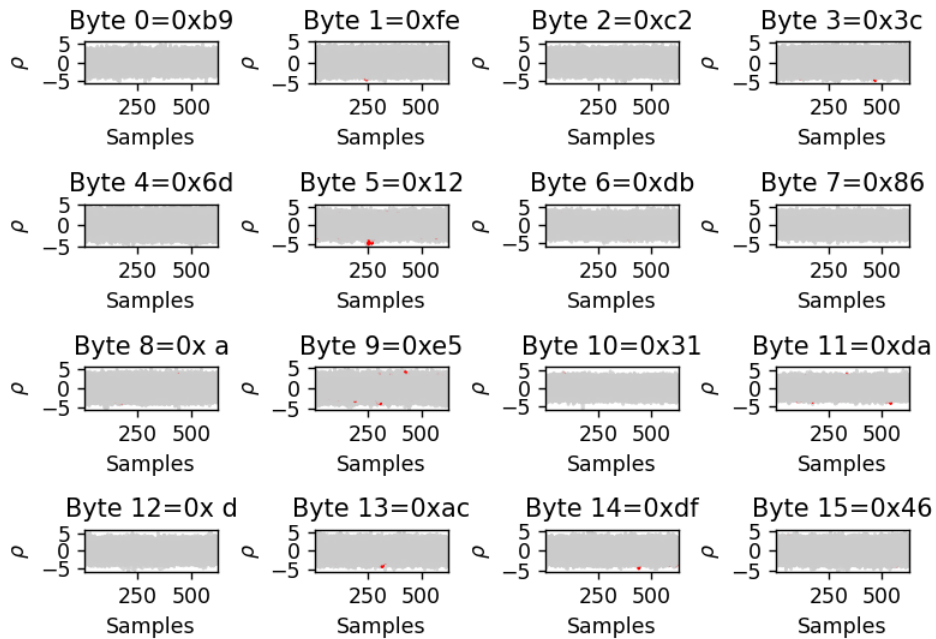


Wrong output bytes : None

Number of correct output bytes : 16/16

DoM Table outputs

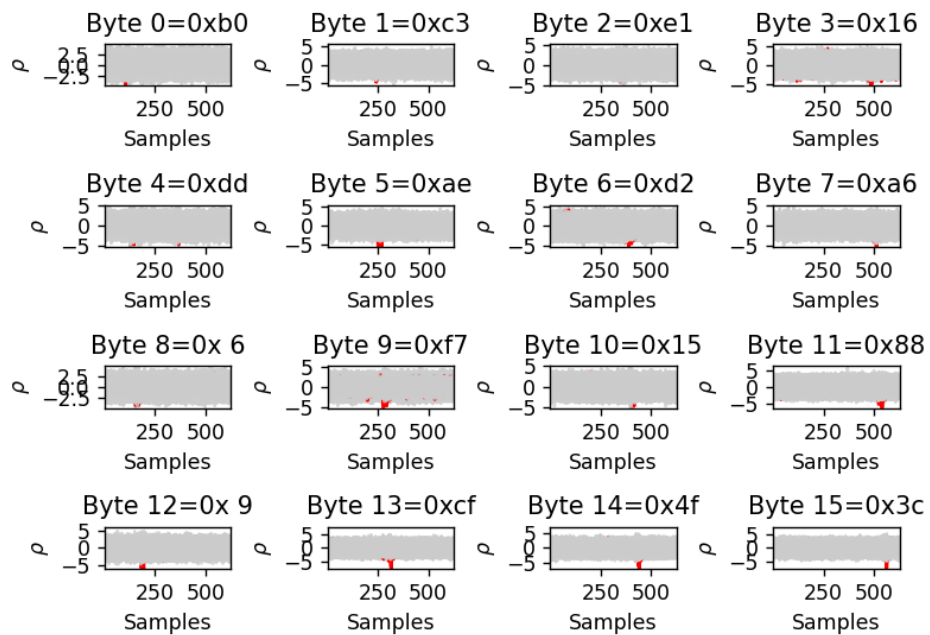
N=100



Wrong output bytes : byte0, byte2, byte3, byte4, byte5, byte6, byte7, byte8, byte9, byte10, byte11, byte12, byte13, byte14, byte15

Number of correct output bytes : 1/16

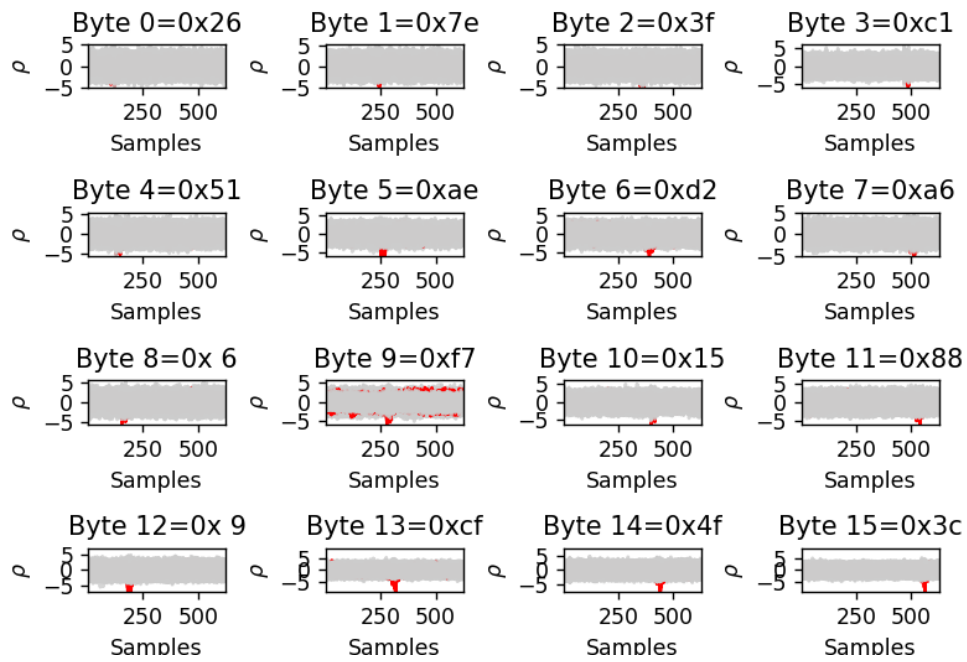
N=400



Wrong output bytes : byte0, byte1, byte2, byte4, byte8

Number of correct output bytes : 11/16

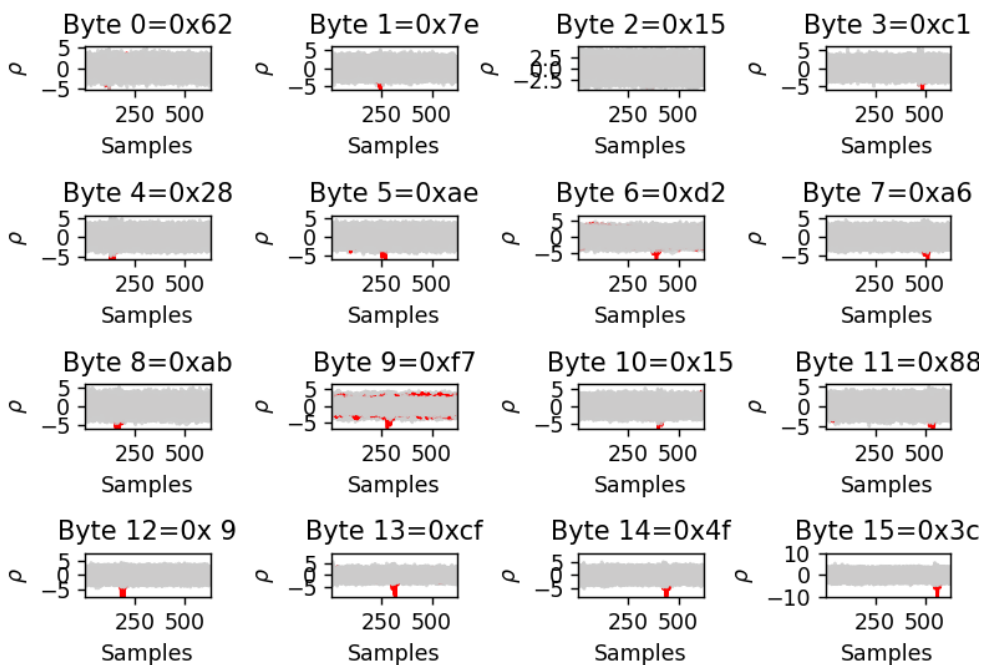
N=500



Wrong output bytes : byte0, byte2, byte3, byte4, byte8

Number of correct output bytes : 11/16

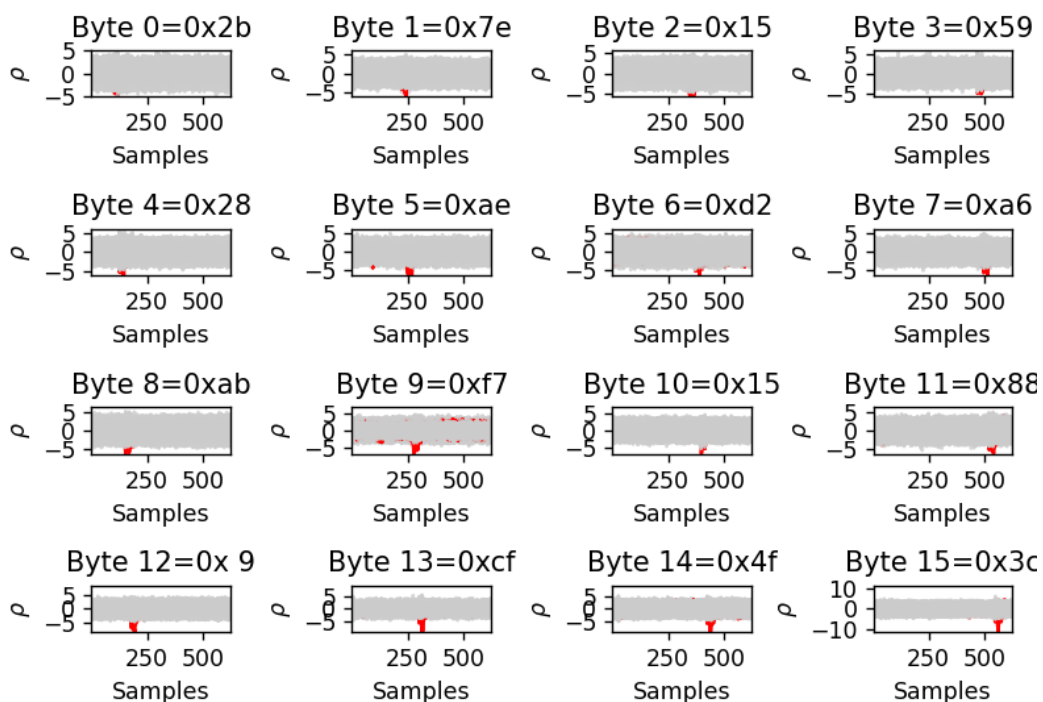
N=600



Wrong output bytes : byte0, byte3

Number of correct output bytes : 14/16

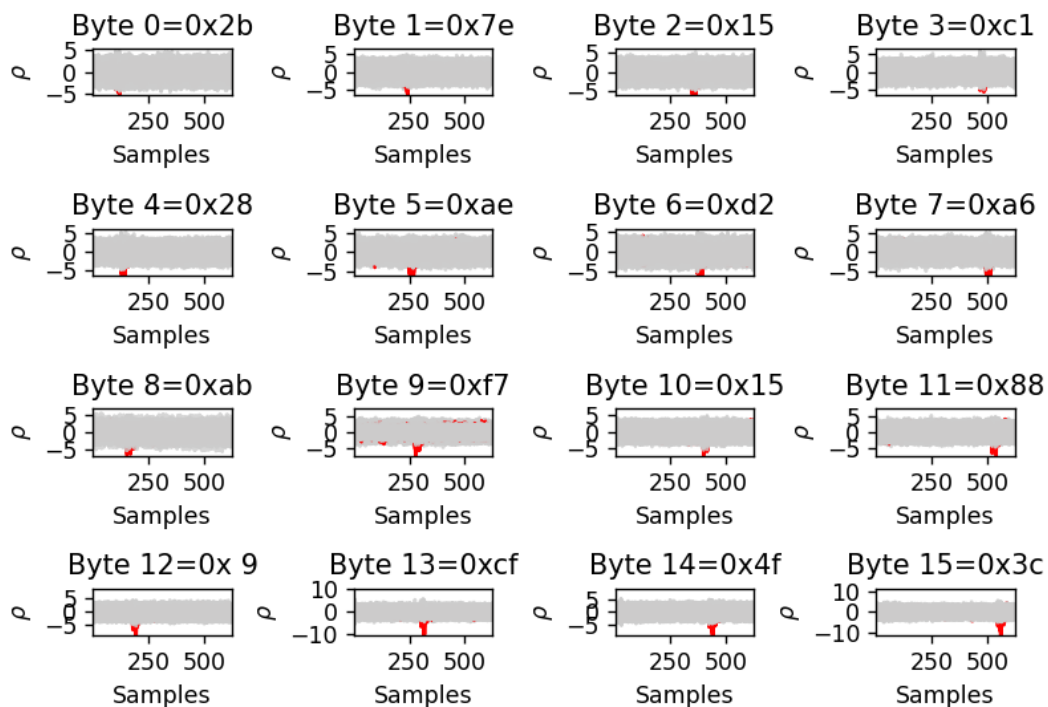
N=700



Wrong output bytes : byte3

Number of correct output bytes : 15/16

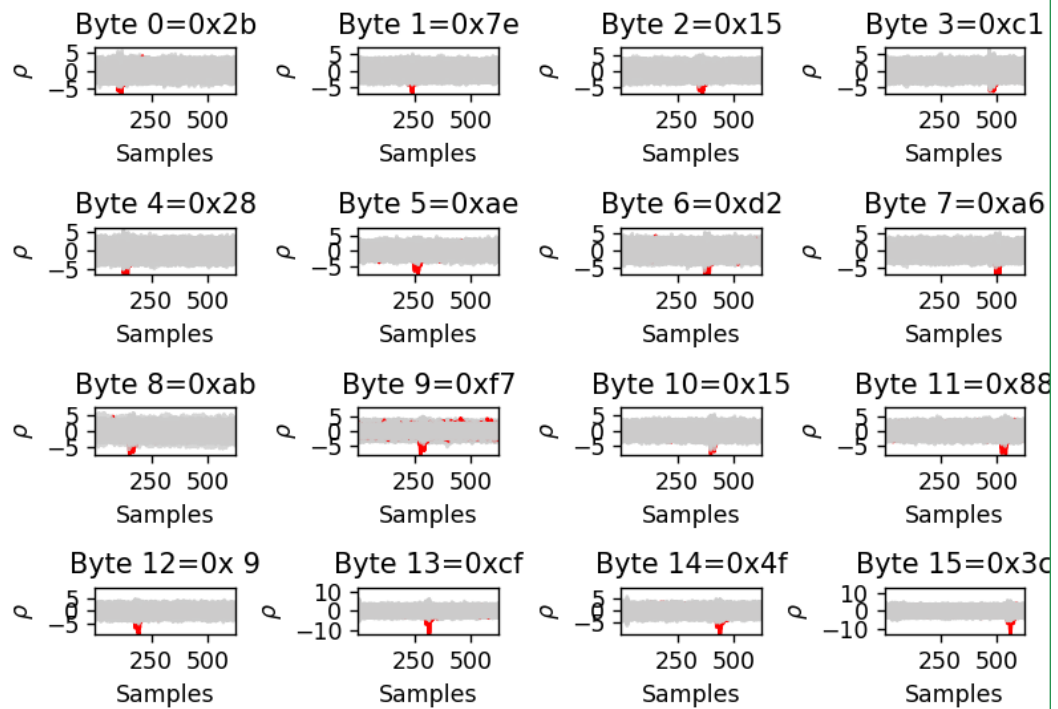
N=800



Wrong output bytes : byte3

Number of correct output bytes : 15/16

N=1000



Wrong output bytes : byte3

Number of correct output bytes : 15/16

Edoardo Diana